MLOps for Heart Disease Classification

End-to-End Machine Learning Development, Experiment Tracking, CI/CD and Deployment

Course: AIMLCZG523 — Machine Learning Operations

Assignment: 01 | Total Marks: 50

Student: Sarth Nitin Khandekar

BITS ID: 2025AF05161

Date: 20 October 2026

Abstract

This project demonstrates an end-to-end, reproducible machine-learning workflow using the processed Cleveland subset of the UCI Heart Disease dataset. It covers dataset acquisition, exploratory data analysis, preprocessing, comparison of Logistic Regression and Random Forest, MLflow experiment tracking, automated tests, model packaging, a FastAPI inference service, Docker containerization, and deployment to a local Kubernetes cluster.

Logistic Regression was selected according to a predefined rule: the highest mean five-fold cross-validation ROC-AUC. It achieved a mean CV ROC-AUC of 0.9025 and a hold-out test ROC-AUC of 0.9665. The API health and prediction endpoints returned HTTP 200 during local Kubernetes verification, and all seven project tests passed during local testing.

This is an educational MLOps demonstration, not a clinically validated medical device. Predictions and confidence values must not be used for diagnosis or medical decisions.

1. Problem Statement, Scope and Dataset

1.1 Problem statement

The assignment objective is to build a binary classifier that estimates whether the dataset label indicates the presence of heart disease, and to demonstrate how such a model can be developed, tested, packaged, deployed and monitored using MLOps practices. The emphasis is on engineering reliability and reproducibility rather than clinical deployment.

1.2 Project objectives

- Acquire the dataset through a repeatable download process and make missing-value handling explicit.
- Inspect feature distributions, data quality, missingness and target-class balance before modelling.
- Compare at least two classification approaches under a consistent evaluation procedure.
- Track experiment parameters, metrics and artifacts so model-selection decisions can be traced.
- Package preprocessing and prediction together to keep inference transformations consistent with training.
- Automate code checks and tests, containerize the API, and verify deployment endpoints.
- Document limitations, setup steps, architecture and evidence for review.

1.3 Dataset source and target mapping

Dataset source: UCI Machine Learning Repository, Heart Disease dataset. The acquisition script downloads the archive and extracts processed.cleveland.data. The selected file contains 13 input features and one target column. The dataset uses '?' for some missing values; these are treated as missing and imputed inside the modelling pipeline.

The original target has values 0 through 4. For this assignment it is converted to a binary target: original value 0 maps to class 0 (absence), while original values 1–4 map to class 1 (presence). This transformation changes the task from distinguishing five levels to predicting a binary outcome, so the report's metrics apply to this binary definition.

Item	Project setting
Dataset	UCI Heart Disease — processed Cleveland subset
Input columns	13 features
Target transformation	0 → 0; 1–4 → 1
Train/test split	80% / 20%, stratified
Random state	42
Validation	Five-fold stratified cross-validation on training partition

Source URL: https://archive.ics.uci.edu/dataset/45/heart+disease

2. Data Acquisition and Exploratory Data Analysis

2.1 Acquisition and data quality

Data acquisition is automated rather than relying on a manually copied local file. The downloader retrieves the UCI archive and extracts the processed Cleveland data file. This supports repeatable setup on another machine, provided the source remains available and the environment has network access. Data parsing treats '?' tokens as missing values so that they can be handled explicitly instead of being interpreted as valid measurements.

Initial data checks examine the dataset dimensions, column data types, missing-value counts, target distribution and descriptive statistics for numerical features. These checks are important because a successful file download alone does not establish that the data has been parsed correctly. Schema and missingness checks help reveal unexpected changes before model training.

2.2 EDA performed

- Dataset shape and feature data types were inspected to understand the table structure.
- Missing values were checked before preprocessing decisions were applied.
- Target-class distribution was examined after defining the binary label.
- Numerical feature summaries and distributions were reviewed to understand ranges and spread.
- Relationships between input variables and the target were considered as part of exploratory analysis.

2.3 How EDA informs the pipeline

EDA is used to guide data preparation, not to make claims about clinical causality. Missing values motivate imputation. Different numerical scales motivate standardization for Logistic Regression. Coded categorical variables are treated as categories and one-hot encoded rather than being assumed to have a meaningful numeric distance. The target mapping is stated explicitly so that the meaning of a positive prediction is unambiguous.

The current report does not include a verified table of exact dataset statistics or embedded EDA plots. Those values and figures should only be added from the actual notebook outputs or saved artifacts; they should not be estimated or fabricated for the sake of presentation.

2.4 EDA limitations

The Cleveland subset is relatively small, and a single dataset cannot represent every population or clinical setting. Apparent relationships in an exploratory sample may be unstable. EDA findings therefore support the modelling workflow but do not establish clinical conclusions. The repository's notebooks/scripts and saved outputs should be used as the source of truth for any numerical descriptive statistics included in a final submission.

3. Feature Engineering and Model Development

3.1 Preprocessing design

The project uses a scikit-learn preprocessing pipeline so that transformations are learned from the training data and applied consistently during validation and inference. Numerical measurements are imputed and standardized. Coded clinical categories are imputed with their most frequent observed value and one-hot encoded. Combining these steps with the estimator helps prevent the deployed service from applying a different preprocessing sequence than the one used during training.

3.2 Avoiding data leakage

Preprocessing is fitted inside the cross-validation training fold rather than on the full dataset before cross-validation. This matters because statistics used for imputation, scaling and category encoding should not be learned from the validation fold. The same principle applies to the held-out test set: it is reserved for final evaluation and should not influence model selection or fitted preprocessing parameters.

3.3 Models compared

Model	Role in experiment	Selection considerations
Logistic Regression	Class-weighted linear classification baseline	Provides a comparatively simple, interpretable decision function; scaling is useful.
Random Forest	Tree-based ensemble comparator	Can capture nonlinear relationships and interactions without requiring feature scaling.

Both models use the same binary target, stratified split and five-fold stratified cross-validation strategy. Using a common evaluation setup makes the comparison more meaningful than comparing results obtained from different splits.

3.4 Reproducibility controls

- Random state 42 is used for the stratified 80:20 split.
- Five-fold stratified cross-validation is performed on the training partition.
- Preprocessing and the classifier are packaged together in a pipeline.
- Model selection follows the declared rule of highest mean CV ROC-AUC.
- Evaluation metrics and supporting artifacts are recorded through the experiment workflow.

The selected model is not chosen solely by training accuracy. Cross-validation gives a view of performance across several partitions of the training data, while the held-out test set provides a separate estimate on data not used to fit the final model. Neither approach guarantees performance on a future population.

4. Evaluation and Model Selection

4.1 Hold-out test results

Model	Accuracy	Precision	Recall	Test ROC-AUC	Mean CV ROC-AUC
Logistic Regression	0.8689	0.8125	0.9286	0.9665	0.9025
Random Forest	0.8852	0.8182	0.9643	0.9513	0.8975

4.2 Cross-validation summary

Model	CV accuracy mean	CV accuracy std. dev.	CV precision mean	CV recall mean
Logistic Regression	0.8552	0.0238	0.8731	0.8099
Random Forest	0.8016	0.0169	0.7831	0.7917

4.3 Metric interpretation

Accuracy is the proportion of all predictions that are correct. Precision is the proportion of predicted positives that are positive in the reference labels. Recall is the proportion of actual positives that the classifier identifies. ROC-AUC measures ranking discrimination across thresholds; it is not the same as accuracy and does not itself identify a suitable operating threshold.

On the hold-out test set, Logistic Regression achieved 86.89% accuracy, 81.25% precision, 92.86% recall and 0.9665 ROC-AUC. Random Forest had somewhat higher test accuracy (88.52%) and recall (96.43%), while Logistic Regression had the higher mean CV ROC-AUC (0.9025 versus 0.8975).

4.4 Selection decision

The predefined selection rule was to choose the model with the highest mean five-fold CV ROC-AUC. Logistic Regression was therefore selected. The difference in mean CV ROC-AUC is small (0.0050), so it should not be interpreted as proof of a large or universally reliable advantage. The selected metric and rule are documented to make the decision traceable.

Because the dataset is small, measured performance may vary with different splits. The evaluation is an educational benchmark and is not evidence of clinical validity. A real medical application would require independent external validation, calibration assessment, threshold selection with domain experts, bias analysis, governance and regulatory review.

5. Experiment Tracking with MLflow

5.1 Purpose

Experiment tracking records the context and results of model-development runs so that the team can inspect what was trained, how it was evaluated and why a model was selected. In this project, MLflow is used for the heart-disease-assignment experiment and records parameters, evaluation metrics and supporting artifacts for Logistic Regression and Random Forest.

5.2 Recorded experiment information

- Experiment name: heart-disease-assignment.
- Evaluation configuration: stratified 80:20 train-test split, random state 42, and five-fold stratified CV on the training partition.
- Metrics include accuracy, precision, recall, ROC-AUC, cross-validation means and accuracy standard deviation.
- Artifacts for the selected Logistic Regression run include a classification report, confusion-matrix plot and ROC curve.
- Local tracking database: mlflow.db; run artifacts are stored under mlruns.

5.3 Run traceability

Model	Finished run ID	Mean CV ROC-AUC
Logistic Regression	f78ee6b994174177b3f52dd11e7afb31	0.9025
Random Forest	46f89ed1666b401f9e4ea7dc9ebada6e	0.8975

5.4 How tracking supports the decision

The recorded CV ROC-AUC values support the documented selection rule and provide a way to revisit the comparison without relying only on a narrative statement. Confusion-matrix and ROC artifacts add detail beyond a single aggregate score. Run IDs make the experiments easier to identify in the local MLflow tracking store.

The tracking database and artifacts are local project outputs. They are not necessarily visible to a reviewer from the public repository unless they have been committed or separately provided, and this report does not claim that a public MLflow dashboard is hosted. If the assignment reviewer requires direct screenshots of MLflow, capture them from the actual local tracking UI rather than using illustrative or recreated screenshots.

5.5 Reproducibility note

A recorded run helps with traceability but does not by itself guarantee bit-for-bit reproducibility. Exact results can depend on Python and library versions, the availability and contents of the source dataset, and execution environment. The repository's dependency definitions and scripts complement MLflow by describing how to recreate the workflow.

6. Model Packaging and Reproducibility

6.1 Saved inference artifact

The selected Logistic Regression model is saved as a serialized scikit-learn pipeline at models/heart_disease_pipeline.joblib. The pipeline bundles preprocessing and the trained classifier, allowing the API to apply the same imputing, scaling and encoding operations expected by the model. Packaging these steps together reduces the risk of training-serving skew.

The companion file models/metadata.json records the selected model, input features, target mapping, random seed, test split and evaluation metrics. This metadata helps a reviewer understand the model artifact without inferring its settings from the serialized object alone.

6.2 Dependency and project configuration

Project dependencies are declared in requirements.txt. The pyproject.toml file provides project configuration and supports editable installation. Keeping dependency definitions in the repository makes setup more repeatable and helps CI execute the project in a controlled environment.

6.3 Reproduction workflow

1. Create and activate a Python virtual environment.
2. Install the dependencies declared by the project.
3. Install the package in editable mode if required by the repository setup.
4. Run the dataset acquisition and training workflow using the project's documented scripts.
5. Run the automated tests before building or serving the API.
6. Use the saved pipeline artifact and configured MODEL_PATH when starting the API.

The exact script names and command-line options should be taken from the repository README and project files. The list above describes the workflow stages and is not intended to replace repository-specific instructions.

6.4 Limits of reproducibility

The pipeline and metadata provide important reproducibility controls, but results may still depend on the Python version, package versions, data source availability and operating environment. For stronger production reproducibility, dependencies could be pinned, artifacts versioned, and the model promoted through a registry with explicit validation gates. These are possible future improvements, not claims about functionality already implemented.

7. CI/CD and Automated Testing

7.1 Automated workflow

The repository uses GitHub Actions to automate checks. The configured workflow runs Ruff code-quality checks, executes the pytest test suite, and performs dataset acquisition and a model-training smoke test. The goal is to detect common failures early and ensure that changes do not silently break important project behavior.

7.2 Testing outcome

All seven project tests passed during local testing. This result refers to the tests executed at that time; it should not be read as a guarantee that every possible input or deployment environment has been tested. The CI workflow is configured to run the checks on GitHub, and its current run status should be verified from the repository's Actions tab before final submission.

7.3 Why the checks matter

Check	Purpose	Evidence to retain
Ruff	Detects configured style and linting issues.	GitHub Actions run or local command output.
Pytest	Checks implemented data/model/API behavior covered by tests.	Test summary showing pass/fail counts.
Training smoke test	Checks that data acquisition and a training path can execute.	Successful workflow log.
Workflow failure behavior	A code or test failure should fail the workflow rather than appear successful.	Actions run showing job status and logs.

7.4 Suggested review before submission

- Open the repository Actions tab and confirm the latest relevant workflow completed successfully.
- Ensure the workflow YAML is committed on the main branch.
- Check that test files are present in the repository's test/ directory.
- Retain a genuine screenshot of the successful workflow run if required by the report rubric.
- Do not claim a workflow passed on GitHub unless the Actions page confirms that status.

7.5 CI/CD scope

The workflow provides automated quality checks and a training smoke test. It should not be described as a complete cloud release pipeline unless automated image publishing and deployment steps are actually configured. This distinction keeps the report aligned with the implementation and makes future extension work clear.

8. FastAPI, Docker, Kubernetes and Monitoring

8.1 Prediction API

The FastAPI service exposes /health and /predict. The health endpoint checks service/model readiness at the level implemented by the application. The prediction endpoint accepts JSON input and returns a prediction, class probabilities, a confidence score and an educational-use disclaimer. The response also includes an X-Process-Time-ms header.

8.2 Request logging

Request middleware logs the HTTP method, request path, response status code and processing duration. These logs provide a basic operational view of incoming requests and endpoint outcomes. They are useful for troubleshooting but do not amount to a full metrics dashboard or production-grade drift-monitoring system.

8.3 Containerization

The API is packaged as a Docker image so the runtime and application can be executed in an isolated container. The project was tested locally, and Docker packaging forms the deployment unit used by the local Kubernetes setup. The container build/test proof required by the assignment should be retained as command output or screenshots from the actual environment.

8.4 Kubernetes deployment

The API was deployed to a local Kubernetes cluster using a Deployment and NodePort Service. At verification time, the Deployment reported one available replica and the running pod had zero restarts. The /health endpoint returned HTTP 200 OK, and a POST request to /predict also returned HTTP 200 OK through Kubernetes port forwarding.

Verification item	Recorded result
Deployment	One available replica at verification time
Pod restarts	Zero at verification time
GET /health	HTTP 200 OK
POST /predict	HTTP 200 OK with prediction response
Local port-forward URL	http://localhost:18001 (temporary session)

The port-forward URL is local to the machine and only works while the forwarding process is running. It is not a public deployment URL. To demonstrate the service later, restart the port-forward using the repository's actual Kubernetes instructions and submit the corresponding access instructions.

8.5 Monitoring limitations

Current monitoring consists of request logs and response timing. The project does not claim production-grade drift detection, a Prometheus/Grafana dashboard or clinical monitoring. Future improvements could include service-level metrics, alerting, structured log aggregation, input validation monitoring and drift analysis, with privacy and data governance considered before collecting operational data.

9. Architecture, Setup, Conclusion and References

9.1 End-to-end architecture

The implemented workflow can be summarized as:

UCI Heart Disease dataset
↓ Download and normalize
EDA and data-quality checks
↓
Train/test split + scikit-learn preprocessing pipeline
↓
Logistic Regression and Random Forest → five-fold CV + hold-out evaluation
↓
MLflow experiment runs and evaluation artifacts
↓
Serialized joblib pipeline + metadata
↓
FastAPI /health and /predict + request logs
↓
Docker container
↓
Kubernetes Deployment + NodePort Service

9.2 Local setup and verification

Use the repository README as the authoritative source for exact commands and prerequisites. At a high level, setup requires a supported Python environment, installing requirements, acquiring the dataset, running training/tests, building the Docker image, and applying the Kubernetes manifests. Verify the actual project commands before copying them into submission instructions.

For a live demonstration, show the test results, a successful /health request, and a sample /predict request using the API's expected input schema. Because local port forwarding is temporary, include clear steps for starting it again. Do not publish patient data or real personal health information in example requests.

9.3 Conclusion

This assignment demonstrates a full educational MLOps workflow spanning data acquisition, EDA, preprocessing, model comparison, MLflow tracking, automated testing, model packaging, FastAPI serving, Docker containerization and Kubernetes deployment. Logistic Regression was selected by the stated cross-validation ROC-AUC rule. Local tests and endpoint checks succeeded during the recorded verification.

Limitations include the small dataset, local-only deployment, absence of production-grade drift monitoring and lack of clinical validation. The reported probability is a model output and should not be interpreted as calibrated clinical certainty. The system must not be used for diagnosis or medical decisions.

9.4 Evidence and references

- Repository: https://github.com/Sarth05/MLOps_Assignment01
- Dataset: UCI Machine Learning Repository, Heart Disease — https://archive.ics.uci.edu/dataset/45/heart+disease
- Kubernetes evidence image referenced in repository: reports/evidence/Kubernetes - Deployment, Health_Check, Prediction.png
- Project artifacts and configuration: requirements.txt, pyproject.toml, models/heart_disease_pipeline.joblib, models/metadata.json, MLflow local store and GitHub Actions workflow.

Submission note: include the repository URL, this report in PDF or DOCX format, a short overall-pipeline demo video, and deployment URL or local access instructions as specified in the assignment. Upload the evidence screenshot separately if the portal asks for it.